

Ledger Sync

Backend Engineering Intern Assignment — Simplify Money
Project Report — Architecture, Domain Analysis, Known Issues & Fixes
Prepared by Chandrama

This report is a status snapshot, not a finished deliverable. Several numbers in the data analysis section (transaction counts, category totals) were produced by manual/AI-assisted review of the corpus and have not yet been confirmed by running the actual parser end to end. They're recorded here as the current working hypothesis, flagged explicitly where relevant, and should be re-verified against real pipeline output before being treated as final.

1. Project Overview

Ledger Sync is the backend service behind Simplify Money's "Track" feature: it reads a user's bank SMS and bank emails, extracts the real financial transactions hidden in that noisy stream, and produces a ledger the user can trust. The assignment hands over a half-finished version of this service, a live incident already open against it, and a 522-line corpus of real-shaped bank messages to run through it.

The service has four jobs:

- Parse raw SMS/email into normalized transactions, deduplicating multi-channel and re-synced evidence of the same event.
- Classify every transaction into exactly one category: SPEND, INCOME, MICRO, or TRANSFER.
- Reconcile: surface anything the ledger can't account for, honestly, rather than force a match.
- Migrate the ledger's storage off SQL onto a document store (DynamoDB or MongoDB), designed around three declared query patterns, plus a backfill and a consistency checker.

Deliverables required: ledger.json, summary.json, reconciliation.json for the given corpus; a fixed and tested incident; a working docker-composed stack with the document store; a Track app teardown; a decision log; an AI-usage disclosure; and an updated CV.

2. Data & Domain Analysis (corpus-a.jsonl)

corpus-a.jsonl contains 522 raw messages across three accounts: two HDFC/ICICI savings accounts (...4821, ...9075) and one HDFC credit card (...3310). Working review of the corpus produced the following taxonomy, used to drive parser and category design:

2.1 Message taxonomy (working analysis, unverified against live parser run)

Category	Est. count	Notes
Transaction-bearing (SMS + email, both banks)	479	HDFC SMS/email + ICICI SMS/email, 4 distinct message templates observed
Non-transaction noise	43	Delivery app noise, OTPs, loan promos, e-mandate pre-notice, balance-inquiry

Adversarial / phishing	5	Spoofed sender headers, threat-of-suspension + fake URL pattern
------------------------	---	---

2.2 Parser traps identified in the corpus

- **Format fragmentation:** each bank has at least two SMS templates (HDFC inline vs. multi-line; ICICI sentence-form vs. a dense Dr/Cr semicolon-delimited form). A parser tuned to only one template per bank will silently under-count — this is the root cause behind Issue #3 in Section 4.
- **Timezone divergence:** some confirmation emails carry a UTC timestamp (+0000) while SMS carries IST (+05:30). A parser that doesn't normalize both to the same instant will split one real transaction into two and double-count it.
- **Credit card vs. savings account fields:** credit card messages report Avl Limit, not Avl Bal, and have no credit leg in this corpus — field-name assumptions that hold for savings accounts don't hold for the card.
- **MICRO is UPI-specific, not amount-specific:** a sub-₹100 UPI debit is MICRO; a sub-₹100 card or netbanking debit is ordinary SPEND. Categorizing by amount alone misclassifies these.
- **Self-transfer vs. external inbound credit:** a same-holder-name inbound credit with no matching outbound leg anywhere in the ledger (e.g. an external NEFT labeled 'SELF') is INCOME, not TRANSFER — only a matched debit/credit pair across the user's own tracked accounts, close in time and equal in amount, qualifies as a transfer leg.
- **Pre-notifications aren't transactions:** e-mandate 'will be deducted' notices describe a future debit, witnessed separately by the real debit message later. Treating the notice itself as a transaction double-counts recurring charges.

2.3 Category rules applied

Category	Rule
SPEND	Money left the user and is gone. Includes card/netbanking debits regardless of amount.
INCOME	Money arrived and is the user's. Includes external inbound credits, even if self-labeled.
MICRO	A UPI debit of ₹100 or less specifically. Listed individually in ledger.json; rolled up to count + total in summary.json —
TRANSFER	One leg of the user moving money between their own tracked accounts. Excluded from both spend and income in sur

3. Architecture & Domain Model

3.1 What a transaction is

A **RawMessage** (one line of the corpus) is not a transaction — it's an external witness testifying that a transaction occurred. A **Transaction** is the underlying economic event: an amount and direction, on a specific account, certified by the bank at a specific instant in IST. One transaction can be evidenced by more than one message (SMS + confirmation email, or a re-synced duplicate upload).

3.2 Transaction identity key

message_id is rejected as an identity key — it's an artifact of the phone's upload/sync process, not of the underlying transaction, so the same economic event can arrive under several different message_ids. Identity is instead a composite of what the bank itself certifies:

```
txn_key = account_last4 + occurred_at (normalized to a UTC instant) + direction + amount
```

Rejected alternatives and why:

- **Merchant string in the key** — rejected. The same event's merchant field varies across channels (SMS: 'UPI/WATER CAN.' vs. email: 'Merchant: WATER CAN VENDOR'), which would fragment one transaction into duplicates.
- **message_id or a hash of the message body** — rejected. Defeats deduplication of exact re-transmissions and multi-channel confirmations by design.
- **Synthetic auto-increment ID** — rejected. Breaks replay idempotency: re-running a failed batch would mint new IDs and corrupt downstream reconciliation.

3.3 Idempotency by construction

Ingestion maintains an identity map keyed by the transaction key described above. Each parsed message either creates a new entry or merges into an existing one (adding its message_id to the evidence set, and filling in the merchant field only if not already populated). No code path creates a duplicate row. Re-ingesting the same file any number of times, or a file that overlaps one already processed, converges to the same ledger — this satisfies the assignment's re-run requirement structurally, rather than through a separate dedup pass bolted on afterward.

3.4 Document store design (three declared query patterns)

The document shape is designed around the three queries the service actually makes, rather than a normalized relational shape ported as-is into a document store:

- **One account's transactions for one month, newest first** — partition/hash key on account + year-month, sort key on occurred_at descending, so this is a single indexed range read, not a scan.
- **Running totals per category for an account** — maintained as a denormalized rollup document per account (and optionally per account+month), updated alongside each transaction write rather than computed by aggregating the full transaction set on every read.
- **Given a message id, which transaction did it produce** — a secondary index (or a small lookup collection) mapping message_id → transaction key, since the primary transaction document is keyed by the transaction identity, not by any one of its evidencing message_ids.

At 100,000 synthetic transactions, each of the three queries should report examined-vs-returned counts close to 1:1 (ScannedCount $\approx$ Count in DynamoDB, or totalDocsExamined $\approx$ nReturned in MongoDB). A query that requires scanning meaningfully more than it returns indicates the document shape doesn't yet match the access pattern and needs revisiting — this check has not yet been run against a live store.

4. Known Issues & Fixes

Current state, from IDE static analysis (7 warnings/TODOs across 4 files), reviewed and grouped into 5 real issues below in suggested fix order.

1. IcciciSmsParser.java — only one ICICI SMS template handled (Ln 14)

HIGH — silent data loss

Issue: Matches only the sentence-form ICICI template ('Dear Customer, Acct XX... is debited with'). The denser Dr/Cr, semicolon-delimited ICICI template is not handled and is either discarded or throws upstream. Every message in that shape is a missing transaction, which violates the assignment's 'every real transaction appears exactly once' requirement and will also fail against the unseen grading corpus.

Fix: Detect template by delimiter/keyword shape before applying a pattern (semicolon+Dr/Cr vs. sentence form), rather than adding a second regex. Route anything from a known bank sender that matches no known template into reconciliation.json as unparsed, instead of silently dropping it.

2. Reports.java — MICRO summed into spend total (Ln 42)

HIGH — contradicts summary.json spec

Issue: MICRO transactions are still added to the same running total as SPEND, inflating the spend figure by every rolled-up UPI micro-debit.

Fix: Branch on category at aggregation time: SPEND → spend total; MICRO → micro_count/micro_total only, never also into spend. Check for a second rollup path if one exists elsewhere in the report builder.

3. Reports.java — TRANSFER counted as both spend and income (Ln 45)

HIGH — the trap the spec names directly

Issue: Both legs of an internal self-transfer still flow into the general accumulator, inflating both spend and income by the full transfer amount, which will not match the checkpoint totals.

Fix: Exclude TRANSFER from both spend and income in summary.json, same accumulator fix shape as MICRO. Verify per-account transfer totals against corpus-a-totals.json after the fix.

4. Backfill.java — source and target fields never read (Ln 15–16)

HIGH — migration does nothing

Issue: The SQL source and document-store target are held as fields but nothing in the class calls either — backfill currently moves zero rows.

Fix: Implement the read–transform–write loop explicitly against source and target: page through SQL, map to the document shape, upsert keyed by transaction identity (never an autoincrement id), and track a checkpoint so a resume after partial failure doesn't reprocess into duplicates.

5. ConsistencyChecker.java — sql and documents fields never read (Ln 16–17)

HIGH — nothing is actually being compared

Issue: Same unused-field pattern as Backfill; as written this class has no data from either side to diff, so it cannot detect the deliberately corrupted document store it will be tested against.

Fix: Load matching pages from both sides keyed by transaction identity and diff field-by-field (amount, direction, category, occurred_at, source_message_ids), emitting exactly which field diverged on which transaction — row-count parity alone is explicitly called out as insufficient.

Suggested fix order

Parser first (data loss upstream invalidates everything downstream), then the two Reports.java accumulator fixes (small, mechanical, directly checkable against corpus-a-totals.json), then Backfill, then ConsistencyChecker last — no point diffing two stores until both sides are known to be correct.

5. Incident — INC-2026-09-11

A ₹5 debit for 'WATER CAN' is being displayed to the user as ₹92,213.10 — which is the account's available balance quoted later in the same SMS, not the transaction amount.

Working hypothesis on root cause

The amount-extraction regex requires two decimal places (matching patterns like Rs . 5, 000 . 00). Whole-rupee amounts written without paise (Rs . 5) don't match, so the parser keeps scanning the message body and locks onto the next decimal-formatted number it finds — the available balance. This is a working hypothesis based on the message shape and needs confirmation against the actual regex once the code is inspected directly.

Why the existing suite likely stayed green

If the amount-parsing tests only exercise amounts already carrying two decimal places, there's no fixture in the suite that would surface a whole-rupee amount colliding with a decimal balance in the same message — a structural gap in test coverage, not a logic error the tests were checking for and missed.

Required before this is closed

- Reproduce against the real corpus (not just the one example) and count every message affected.
- Write a regression test that fails pre-fix and passes post-fix.
- Fix the amount parser to accept whole-rupee amounts without treating a later decimal figure as a fallback match.
- Five-line incident note for the incident channel: what broke, how it was found, who was affected, why it can't recur.

6. Current Status

Done / drafted

- Corpus taxonomy and category rules worked out from manual review (Section 2) — pending verification by an actual parser run.
- Transaction identity key and idempotency design (Section 3.2–3.3).
- Document store access-pattern design (Section 3.4) — not yet implemented or benchmarked.
- Root-cause hypothesis for the incident (Section 5) — not yet confirmed against actual code.

Not yet done

- All 5 code issues in Section 4 are still open in the IDE — none confirmed fixed yet.
- No full pipeline run against corpus-a.jsonl has been executed and diffed against corpus-a-totals.json.
- Backfill and ConsistencyChecker are unimplemented (fields unused).
- Document store not yet stood up; docker compose and ./verify.sh not yet confirmed working end to end.
- Task 0 (referrals + feedback one-pager) and Task 1 (Track app teardown) — require using the live app, not yet done.

- Decision log, AI disclosure with a real before/after example, and unfinished-work list — to be written from what actually happens during the fixes above, not drafted in advance.

This report reflects analysis and design work done ahead of running the actual code. Sections 2 and 5 in particular contain hypotheses that should be confirmed — or corrected — once the parser is run against the real corpus and the incident is reproduced directly.